

- **Lab Title:** Lab 1A: Number Systems, ASCII, Timestamps and Data Representation
- **Assignment Title:** Basic Computer Skills for Digital Forensics
- **Student Name:** Bashir Adam
- **Student ID:** 2025/FWSD/11509
- **Course Name:** SBT-DF201: Basic Computer Skills for Digital Forensics
- **Instructor Name:** Aminu Idris
- **Date of Submission:** 8 Aug, 2026
- **Version:** Version 2.1.0

platform used: Kali Linux

LAB OVERVIEW/OBJECTIVE

Scenario: You are a beginner digital forensics trainee. During an investigation, you receive values from disk sectors, command-line outputs, logs, hashes and memory dumps. Your task is to correctly interpret numbers, text encodings, timestamps and byte order so that evidence is not misunderstood.

Task A - Number Systems and Manual Conversion

The conversion of 101010_2 to decimal manually

2.2 Practical conversion Exercises
① Convert 101010_2 to decimal

$101010_2 \Rightarrow$

$$\begin{array}{rcl} 0 \times 2^0 & = & 0 \\ 1 \times 2^1 & = & 2 \\ 0 \times 2^2 & = & 0 \\ 1 \times 2^3 & = & 8 \\ 0 \times 2^4 & = & 0 \\ 1 \times 2^5 & = & 32 \\ \hline & & 42_{10} \end{array}$$

$\therefore$ The ans 42_{10}

The conversion of $AB2_{16}$ to decimal manually

② Convert $AB2_{16}$ to binary

$AB2_{16} \Rightarrow$

$$\begin{array}{rcl} 2 \times 16^0 & = & 2 \times 1 = 2 \\ B \times 16^1 & = & 11 \times 16 = 176 \\ A \times 16^2 & = & 10 \times 256 = 2560 \\ \hline & & 2738_{10} \end{array}$$

$\therefore$ The ans 2738_{10}

The conversion of 126_{10} to binary manually by division by 2.

③ Convert 126_{10} to binary (2)

$126_{10} \Rightarrow$

2	/ 126	r 0
2	/ 63	r 0
2	/ 31	r 1
2	/ 15	r 1
2	/ 7	r 1
2	/ 3	r 1
2	/ 1	r 1
	0	r 1

$\therefore 126_{10} = 1111110_2$

The conversion of 2738_{10} to hexadecimal manually by division by 16.

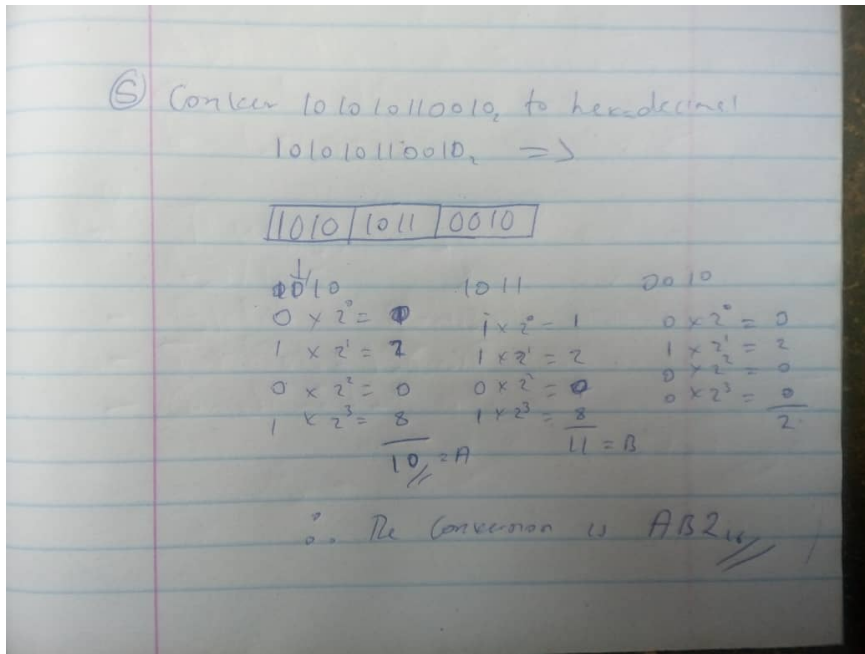
④ Convert 2738_{10} to hexadecimal (16)

$2738_{10} \Rightarrow$

16	/ 2738	
16	/ 172	r 2
16	/ 10	r 11 = B
	1 0	r 10 = A

$\therefore 2738_{10} = 10\ 11\ 2 = AB2_{16}$

The conversion of 101010110010_2 to hexadecimal by grouping bits in fours from the right.



To verify my findings i create a workspace using the command mkdir and view the active directory am in.

```
azad@localhost: ~/cip-b101-lab1a
$ mkdir -p ~/cip-b101-lab1a
cd ~/cip-b101-lab1a
pwd
/home/azad/cip-b101-lab1a
$
```

File	File Description	File Description
And	And	And
And	And	And
And	And	And
And	And	And

1.2 Student Working Folder

```
mkdir -p ~/cip-b101-lab1a
cd ~/cip-b101-lab1a
```

2. Task A- Number Systems and Manual Conversion

This task builds your ability to interpret values written in base 10, base 2 and base 16. Forensic examiners encounter hexadecimal values in memory, disk sectors, file headers, hash outputs and metadata structures.

2.1 Quick Reference Table

Decimal	Binary	Hexadecimal
0	0	0
1	1	1
2	10	2

I start verifying all step by step using the command listed below

```
azad@localhost: ~/cip-b101-lab1a
$ mkdir -p ~/cip-b101-lab1a
cd ~/cip-b101-lab1a
pwd
/home/azad/cip-b101-lab1a

azad@localhost:~/cip-b101-lab1a
$ echo $((2#101010))
42

azad@localhost:~/cip-b101-lab1a
$ echo $((16#AB2))
2738

azad@localhost:~/cip-b101-lab1a
$ echo "obase=2;126" | bc
1111110

azad@localhost:~/cip-b101-lab1a
$ printf "%x"
2738

AB2

azad@localhost:~/cip-b101-lab1a
$
```

This is the python verification

```
azad@localhost: ~/cip-b101-lab1a
$ python
Python 3.13.14 (main, Jun 10 2026, 18:10:12) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print(int('101010', 2))
42
>>> print(int('AB2', 16))
2738
>>> print(int('126', 10))
126
>>> print(int('AB2', 16))
2738
>>> print(bin(126))
'0b1111110'
>>> print(hex(2738))
'0xab2'
>>>
```

Table of summary of my findings

Evidence Value	Given Base	Manual Answer	Command Used to Verify	Screenshot Ref.
101010	Binary	42 ₁₀	echo \$((2#101010))	Screenshot above
AB2	Hexadecimal	2738 ₁₀	echo \$((16#AB2))	Screenshot above
126	Decimal	1111110 ₂	echo "obase=2;126" bc	Screenshot above
2738	Decimal	AB2 ₁₆	printf "%X" " 2738	Screenshot above
101010110010	Binary	AB2 ₁₆	printf "%X" " 2738	Screenshot above

Task B - ASCII, UTF-8 and Hex Evidence Strings

I created text evidence and view as hex by runing all the simple command

```

azad@localhost: ~/cip-b101-lab1a
$ echo -n "Hello" > evidence_text.txt

$ cat evidence_text.txt
Hello

$ xxd evidence_text.txt
00000000: 4865 6c6c 66 00 00 00 00 00 00 00 00 00 00 00 00 00
48656c6c66

$ xxd -p evidence_text.txt
48656c6c66

$
  
```

The screenshot shows a terminal window with the following content:

- Terminal title: azad@localhost: ~/cip-b101-lab1a
- Command: `echo -n "Hello" > evidence_text.txt`
- Command: `cat evidence_text.txt` (Output: Hello)
- Command: `xxd evidence_text.txt` (Output: 00000000: 4865 6c6c 66 00 00 00 00 00 00 00 00 00 00 00 00 00 48656c6c66)
- Command: `xxd -p evidence_text.txt` (Output: 48656c6c66)
- Command: `$` (Prompt)

Overlaid on the terminal is a document titled "Task B - ASCII, UTF-8 and Hex Evidence Strings". The document contains the following sections:

- 2.1 Create Text Evidence and View as Hex**

```

#!/bin/bash
# Create a text file named evidence_text.txt
echo -n "Hello" > evidence_text.txt
  
```
- 2.2 Convert Hex to ASCII**

```

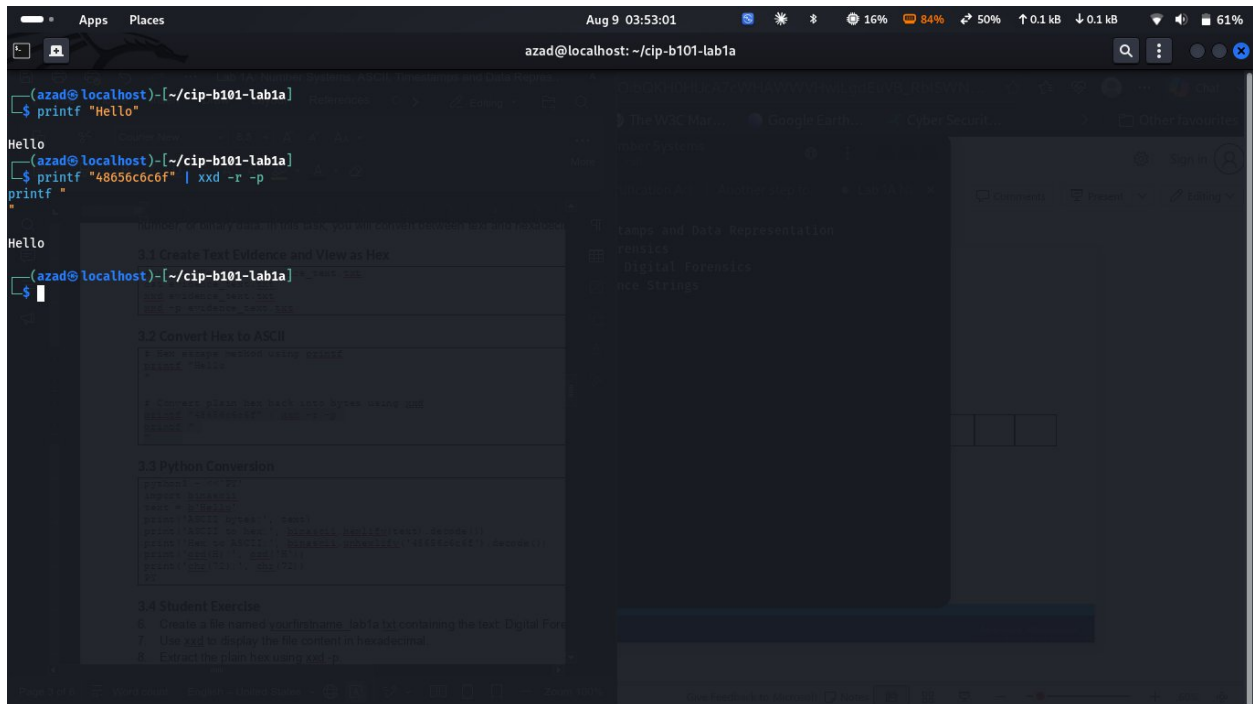
#!/bin/bash
# Get hex string input using xxd
hex="48656c6c66"

# Convert plain hex back into bytes using xxd
echo "$(xxd -r "$hex")" | cat -p
  
```
- 2.3 Python Conversion**

```

python3 -c '
import sys
hex = "48656c6c66"
print(hex)
print(hex[0:4], hex[4:8], hex[8:12], hex[12:16])
print(hex[0:4].decode("hex"), hex[4:8].decode("hex"), hex[8:12].decode("hex"), hex[12:16].decode("hex"))
print(hex[0:4].decode("hex"), hex[4:8].decode("hex"), hex[8:12].decode("hex"), hex[12:16].decode("hex"))
  
```

I Convert Hex to ASCII using printf and xxd with the arggument r&p

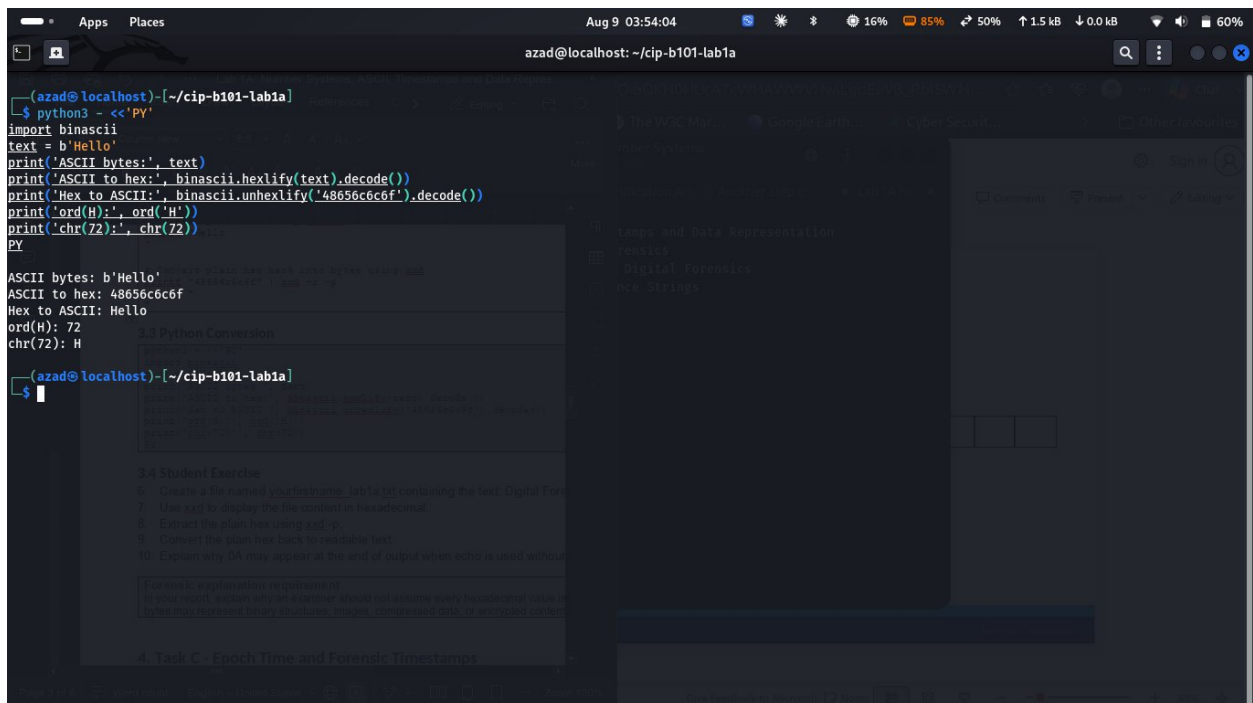


A terminal window titled 'azad@localhost: ~/cip-b101-lab1a' showing the following commands and output:

```
(azad@ localhost)-[~/cip-b101-lab1a]
$ printf "Hello"
Hello
(azad@ localhost)-[~/cip-b101-lab1a]
$ printf "48656c6c66" | xxd -r -p
printf "
"
```

The output shows the string "Hello" and then a blank line after the hex conversion. A document titled "3.1 Create Text Evidence and View as Hex" is visible in the background.

I convert all using Python command



A terminal window titled 'azad@localhost: ~/cip-b101-lab1a' showing the following commands and output:

```
(azad@ localhost)-[~/cip-b101-lab1a]
$ python3 -c <<'PY'
import binascii
text = b'Hello'
print('ASCII bytes:', text)
print('ASCII to hex:', binascii.hexlify(text).decode())
print('Hex to ASCII:', binascii.unhexlify('48656c6c66').decode())
print('ord(H):', ord('H'))
print('chr(72):', chr(72))
PY
ASCII bytes: b'Hello'
ASCII to hex: 48656c6c66
Hex to ASCII: Hello
ord(H): 72
chr(72): H
```

The output shows the string "Hello" and then a blank line after the hex conversion. A document titled "3.1 Create Text Evidence and View as Hex" is visible in the background.

This is a simple student exercise i conduct by using the echo, xxd and printf

```
azad@localhost: ~/cip-b101-lab1a
(azad@localhost)-[~/cip-b101-lab1a]
$ echo -n Digital Forensics Begins With Bytes. > Adam_lab1a.txt
(azad@localhost)-[~/cip-b101-lab1a]
$ xxd -p Adam_lab1a.txt
4469676974616c20466f72656e7369637320426567696e73205769746820
42797465732e
(azad@localhost)-[~/cip-b101-lab1a]
$ xxd Adam_lab1a.txt
00000000: 44 69 67 69 74 61 6c 20 46 6f 72 65 6e 73 69 63  Digital Forensic
00000010: 73 20 42 65 67 69 6e 73 20 57 69 74 68 20 42 79  s Begins With By
00000020: 74 65 73 2e                                         tes.
(azad@localhost)-[~/cip-b101-lab1a]
$ xxd -p -r Adam_lab1a.txt
(azad@localhost)-[~/cip-b101-lab1a]
$ cat Adam_lab1a.txt
Digital Forensics Begins With Bytes.
(azad@localhost)-[~/cip-b101-lab1a]
$ printf "4469676974616c20466f72656e7369637320426567696e73205769746820
42797465732e
" | xxd -r -p
Digital Forensics Begins With Bytes.
(azad@localhost)-[~/cip-b101-lab1a]
$
```

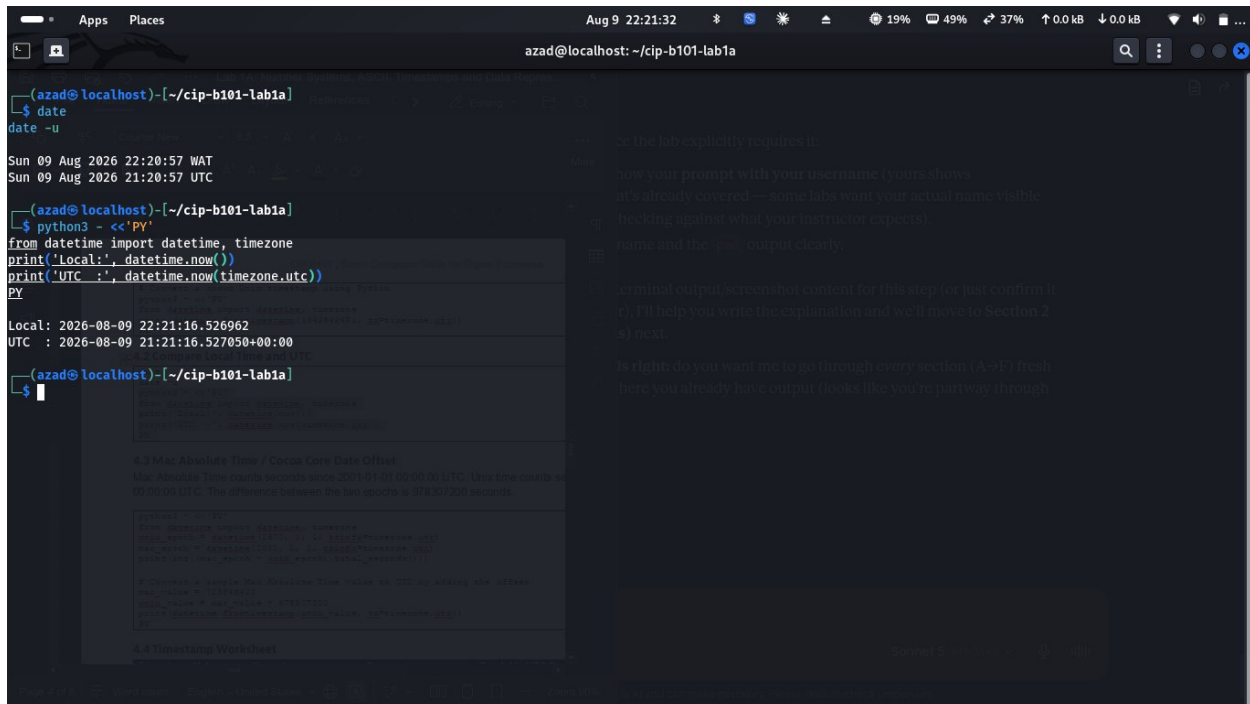
When I use echo without -n, it automatically appends a newline character after the text. In ASCII, a newline (line feed) is byte value 0A in hex. So if I hadn't used -n, the hex dump would show an extra 0a byte tacked on at the very end not part of my actual sentence, just echo's default behavior of terminating output with a newline. That's exactly why I used -n throughout to keep the file's byte content pure and match exactly what I typed, without an invisible extra byte at the end.

Task C - Epoch Time and Forensic Timestamps

By using the timestamps i was able to conversion unix epoch manually and python

```
azad@localhost: ~/cip-b101-lab1a
(azad@localhost)-[~/cip-b101-lab1a]
$ ls
cip-b101-lab1a  Desktop  Music  Proj  school-portal  Videos
ctf-platform  Documents  package  Projects  school-portal.zip
ctf-platform.zip  Downloads  Pictures  Public  Templates
(azad@localhost)-[~/cip-b101-lab1a]
$ cd cip-b101-lab1a
(azad@localhost)-[~/cip-b101-lab1a]
$ ls
Adam_lab1a.txt  evidence_text.txt
(azad@localhost)-[~/cip-b101-lab1a]
$ date +%s
1786272520
(azad@localhost)-[~/cip-b101-lab1a]
$ date -u -d @1786272520
Sun 09 Aug 2026 10:48:40 UTC
(azad@localhost)-[~/cip-b101-lab1a]
$ python3 -c <<Py
from datetime import datetime, timezone
print(datetime.fromtimestamp(1643643491, tz=timezone.utc))
Py
2022-01-31 15:38:11+00:00
(azad@localhost)-[~/cip-b101-lab1a]
$
```


And the i was able to compare local time and UTC by using python and manual method



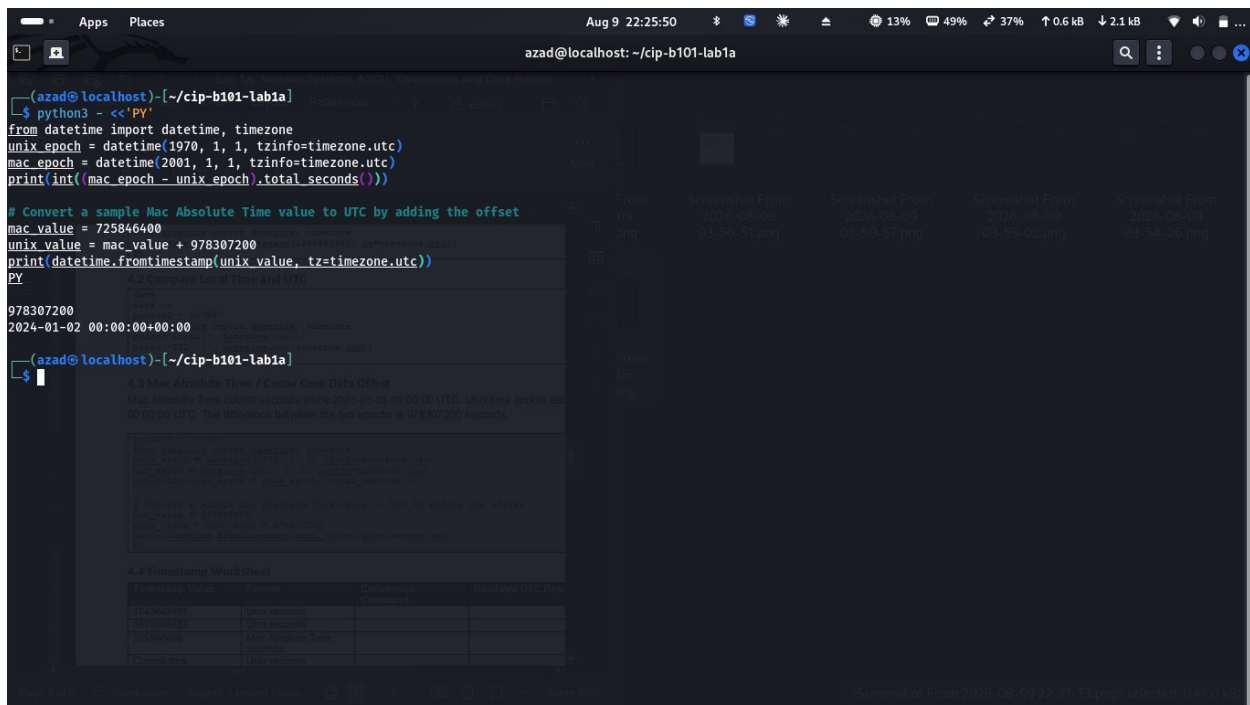
```
(azad@ localhost)-[~/cip-b101-lab1a]
$ date
date -u
Sun 09 Aug 2026 22:20:57 WAT
Sun 09 Aug 2026 21:20:57 UTC

(azad@ localhost)-[~/cip-b101-lab1a]
$ python3 - <<'PY'
from datetime import datetime, timezone
print('local:', datetime.now())
print('UTC :', datetime.now(timezone.utc))
PY
Local: 2026-08-09 22:21:16.526962
UTC : 2026-08-09 21:21:16.527050+00:00

(azad@ localhost)-[~/cip-b101-lab1a]
```

The terminal window shows the user running the `date` command to display local and UTC times. Then, they use `python3` to import `datetime` and `timezone` modules, and print the current local and UTC timestamps. The output shows a local time of 22:21:16.526962 and a UTC time of 21:21:16.527050+00:00.

I also Mac Absolute Time / Cocoa Core Date Offset to see the seconds interval



```
(azad@ localhost)-[~/cip-b101-lab1a]
$ python3 - <<'PY'
from datetime import datetime, timezone
unix_epoch = datetime(1970, 1, 1, tzinfo=timezone.utc)
mac_epoch = datetime(2001, 1, 1, tzinfo=timezone.utc)
print(int((mac_epoch - unix_epoch).total_seconds()))

# Convert a sample Mac Absolute Time value to UTC by adding the offset
mac_value = 725846400
unix_value = mac_value + 978307200
print(datetime.fromtimestamp(unix_value, tz=timezone.utc))
PY
978307200
2024-01-02 00:00:00+00:00

(azad@ localhost)-[~/cip-b101-lab1a]
```

The terminal window shows the user running a `python3` script to calculate the difference between the Mac epoch (2001-01-01) and the Unix epoch (1970-01-01). The result is 978307200 seconds. Then, they use this offset to convert a sample Mac Absolute Time value (725846400) to a Unix timestamp (1725136640) and print the corresponding UTC date and time: 2024-01-02 00:00:00+00:00.

Timestamp Worksheet

Timestamp Value	Format	Conversion Command	Readable UTC Result	Screenshot Ref.
1643643491	Unix seconds	date -u -d @1786272520	Sun 09 Aug 2026 10:48:40 UTC	Pic above
1675008832	Unix seconds	date date -u	Sun 09 Aug 2026 22:33:56 WAT Sun 09 Aug 2026 21:33:56 UTC	Pic above
725846400	Mac Absolute Time seconds	python	978307200	Pic above
Current time	Unix seconds	python	978307200 2024-01-02 00:00:00+00:00	Pic above

Task D - Hashing and the Newline Problem

I created two files that look identical when printed, but differ at the byte level. I used xxd to inspect both files at the byte level. I ran both hashing algorithms against the two files. Even though the visible text is the same, the MD5 and SHA-256 hashes came out completely different for each file proving that hash functions operate on raw bytes, not on what a human sees on screen.

```
(azad@localhost)~/cip-b101-lab1a
$ echo "Hello" > hello_with_newline.txt
$ echo -n "Hello" > hello_without_newline.txt

(azad@localhost)~/cip-b101-lab1a
$ xxd hello_with_newline.txt
xxd hello_without_newline.txt

00000000: 4865 6c6c 6f0a                Hello.
00000000: 4865 6c6c 6f                Hello

(azad@localhost)~/cip-b101-lab1a
$ md5sum hello_with_newline.txt hello_without_newline.txt
sha256sum hello_with_newline.txt hello_without_newline.txt

09f7e02f1290be211da707a266f153b3  hello_with_newline.txt
8b1a9953c4611296a827abf8c47804d7  hello_without_newline.txt
66a045b452102c59d840ec097d59d9467e13a3f34f6494e539ffd32c1bb35f18  hello_with_newline.txt
185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969  hello_without_newline.txt

(azad@localhost)~/cip-b101-lab1a
$
```

Student Exercise

1. Create two files with my first name

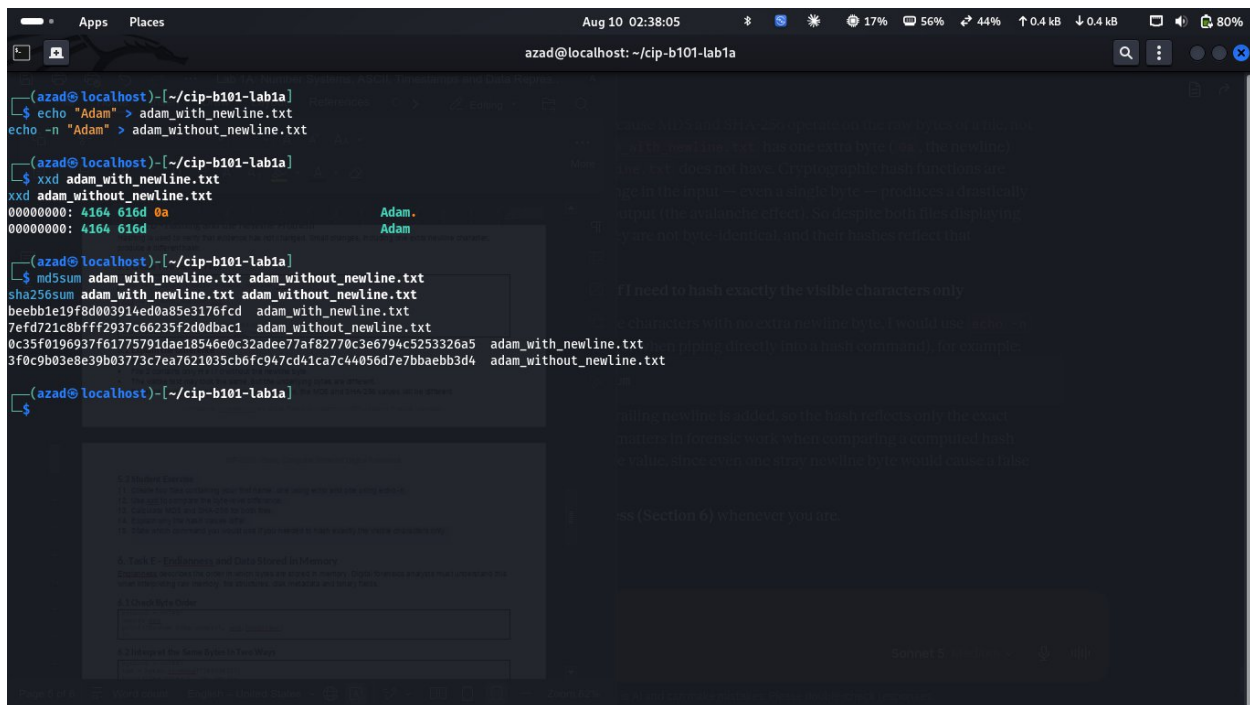
I created two files containing my first name, "Adam" one with echo, which appends a trailing newline by default, and one with echo -n, which suppresses it.

2. Compare byte-level difference with xxd

I run xxd on adam_with_newline.txt shows the hex bytes 41 64 61 6d 0a the letters A-d-a-m followed by 0a, the newline byte. Running xxd on adam_without_newline.txt shows only 41 64 61 6d the same four letters, but with no trailing byte. This confirms the two files differ by exactly one byte at the end.

3. Calculate MD5 and SHA-256 for both

I ran both hashing commands against the two files. Even though both files "look" like they contain just my name, the hash values produced were completely different for each file, for both MD5 and SHA-256.



```
(azad@localhost)-[~/cip-b101-lab1a]
$ echo "Adam" > adam_with_newline.txt
echo -n "Adam" > adam_without_newline.txt

(azad@localhost)-[~/cip-b101-lab1a]
$ xxd adam_with_newline.txt
xxd adam_without_newline.txt
00000000: 4164 616d 0a                Adam.
00000000: 4164 616d                   Adam

(azad@localhost)-[~/cip-b101-lab1a]
$ md5sum adam_with_newline.txt adam_without_newline.txt
sha256sum adam_with_newline.txt adam_without_newline.txt
beebb1e19f8d003914ed0a85e3176fcd  adam_with_newline.txt
7efd721c8bfff2937c66235f2d0dbac1  adam_without_newline.txt
0c35f0196937f61775791dae18546e0c32adee77af82770c3e6794c5253326a5  adam_with_newline.txt
3f0c9b3e8e39b03773c7ea7621035cb6fc947cd41ca7c44056d7e7bbaebb3d4  adam_without_newline.txt

(azad@localhost)-[~/cip-b101-lab1a]
$
```

4. Explain why the hash values differ

The hash values differ because MD5 and SHA-256 operate on the raw bytes of a file, not on the "visible" text. adam_with_newline.txt has one extra byte (0a, the newline) that adam_without_newline.txt does not have. Cryptographic hash functions are designed so that any change in the input even a single byte produces a drastically different, unpredictable output (the avalanche effect). So despite both files displaying "Adam" when opened, they are not byte-identical, and their hashes reflect that difference completely.

5. Which command to use if I need to hash exactly the visible characters only

To hash exactly the visible characters with no extra newline byte, I would use `echo -n` when generating the file (or when piping directly into a hash command), for example: `echo -n "Adam" | md5sum`

The `-n` flag ensures no trailing newline is added, so the hash reflects only the exact visible characters this matters in forensic work when comparing a computed hash against a known reference value, since even one stray newline byte would cause a false mismatch.

Task E - Endianness and Data Stored in Memory

I checked my system's native byte order using Python's `sys.byteorder`. The result, little, I took the same 4 raw bytes (78 56 34 12) and interpreted them two different ways. Read as little-endian, the bytes are reversed before calculating the value, giving 12 34 56 78 in evaluation order, which equals 305,419,896. Read as big-endian, the bytes are evaluated exactly as stored, 78 56 34 12, which equals 2,018,915,346.

```
(azad@ localhost) [~/cip-b101-lab1a]
$ python3 -c <<'PY'
import sys
print('System byte order:', sys.byteorder)
PY
System byte order: little

(azad@ localhost) [~/cip-b101-lab1a]
$ python3 -c <<'PY'
raw = bytes.fromhex('78563412')
print('Raw bytes:', raw.hex())
print('Interpreted as little-endian:', int.from_bytes(raw, 'little'))
print('Interpreted as big-endian:', int.from_bytes(raw, 'big'))
PY
Raw bytes: 78563412
Interpreted as little-endian: 305419896
Interpreted as big-endian: 2018915346

(azad@ localhost) [~/cip-b101-lab1a]
```

The screenshot shows a terminal window with the following content:

System byte order: little

Raw bytes: 78563412

Interpreted as little-endian: 305419896

Interpreted as big-endian: 2018915346

Bytes In Two Ways

6.3 Student Exercise

Hex Value	Little-Endian Decimal	Big-Endian Decimal
01000000		
FF000000		
00000000		
78563412		

Forensic explanation requirement

In your own words, explain how the same four bytes can produce different results.

Student Exercise

Raw Hex Bytes	Little-Endian Decimal	Big-Endian Decimal	Command Used	Explanation
01000000	1	16777216	<code>int.from_bytes(bytes.fromhex('01000000'),'little') / 'big'</code>	Little-endian reads the first byte as the least significant,
FF000000	255	4278190080	<code>int.from_bytes(bytes.fromhex('FF000000'),'little') / 'big'</code>	Little-endian treats FF as the smallest position 255.
00000100	65536	256	<code>int.from_bytes(bytes.fromhex('00000100'),'little') / 'big'</code>	The nonzero byte 01 sits third from the left.
78563412	305419896	2018915346	<code>int.from_bytes(bytes.fromhex('78563412'),'little') / 'big'</code>	Little-endian reverses the byte order before evaluating,

Task F - Mini Forensic Interpretation Case

Evidence Item	Value	Student Interpretation	Command/Screenshot Ref.
EVID-001	48656c6c6f204943444641	Decoded ASCII text: "Hello ICDFA"	<code>printf "48656c6c6f204943444641" xxd -r -p</code>
EVID-002	01001000 01101001	Decoded ASCII text: "Hi"	<pre>python3 - <<'PY' b1 = int('01001000', 2) b2 = int('01101001', 2) print('First byte:', b1, '->', chr(b1)) print('Second byte:', b2, '->', chr(b2)) print('Decoded text:', chr(b1) + chr(b2)) PY</pre>
EVID-003	1675008832	Unix timestamp → 2023-01-29 16:13:52 UTC	<code>date -u -d @1675008832</code>
EVID-004	725846400 (Mac Absolute Time)	Time)Mac time → Unix 1704153600 → 2024-01-02 00:00:00 UTC	<pre>python3 - <<'PY' from datetime import datetime, timezone mac_value = 725846400 unix_value = mac_value + 978307200 print(datetime.fromtimestamp(unix_value, tz=timezone.utc)) PY</pre>
EVID-005	78563412 as little-endian and big-endian	Little-endian = 305,419,896; Big-endian = 2,018,915,346	<pre>python3 - <<'PY' raw = bytes.fromhex('78563412') print('Little-endian:', int.from_bytes(raw, 'little')) print('Big-endian :', int.from_bytes(raw, 'big')) PY</pre>

```
Aug 10 03:17:04
azad@localhost: ~/cip-b101-lab1a

(azad@localhost)~/~/cip-b101-lab1a
$ printf "48656c66f204943444641" | xxd -r -p
Hello ICDFA

(azad@localhost)~/~/cip-b101-lab1a
$ python3 -c '<<'PY'
b1 = int('01001000', 2)
b2 = int('01101001', 2)
print('First byte:', b1, '->', chr(b1))
print('Second byte:', b2, '->', chr(b2))
print('Decoded text:', chr(b1) + chr(b2))
PY
First byte: 72 -> H
Second byte: 105 -> i
Decoded text: Hi

(azad@localhost)~/~/cip-b101-lab1a
$ date -u -d @1675008832
Sun 29 Jan 2023 16:13:52 UTC

(azad@localhost)~/~/cip-b101-lab1a
$ python3 -c '<<'PY'
from datetime import datetime, timezone
mac_value = 725846400
unix_value = mac_value + 978307200
print(datetime.fromtimestamp(unix_value, tz=timezone.utc))
PY
2024-01-02 00:00:00+00:00

(azad@localhost)~/~/cip-b101-lab1a
$ python3 -c '<<'PY'
raw = bytes.fromhex('78563412')
print('Little-endian:', int.from_bytes(raw, 'little'))
print('Big-endian:', int.from_bytes(raw, 'big'))
PY
Little-endian: 305419896
Big-endian : 2018915346

(azad@localhost)~/~/cip-b101-lab1a
```

EVID-001 decodes from hex to "Hello ICDFA" plain text stored as hex bytes.

EVID-002 decodes from binary to "Hi" showing binary and hex just represent the same bytes differently.

EVID-003 is a Unix timestamp **2023-01-29 16:13:52 UTC**, using date -u -d @1675008832 (UTC used deliberately for a consistent timeline). EVID-004 is a Mac Absolute Time value, which needs the 978,307,200-second offset added before converting **2024-01-02 00:00:00 UTC**; using the wrong epoch here would misplace the event by over 23 years.

EVID-005 shows the same 4 bytes giving two different values depending on byte order: **305,419,896** (little-endian) vs **2,018,915,346** (big-endian) neither is wrong on its own; it depends on the source format's defined byte order.

For usefulness: EVID-003 and EVID-004 support **timeline analysis** (though EVID-004 needed epoch correction first). EVID-001 and EVID-002 support **text/content interpretation**. EVID-005 supports neither directly it's a reminder that raw numeric fields (sizes, offsets, IDs) must never be read without confirming byte order first.

Conclusion

This lab showed me that number systems, encodings, and timestamps aren't just theory they're core forensic skills. A hex string, a binary sequence, or a raw epoch number can be completely misread if you assume the wrong base, encoding, or epoch reference. Doing the manual conversions helped me actually understand what forensic tools are doing behind the scenes when they show memory addresses, offsets, or sector data. Working through the ASCII/hex exercises showed how easily readable text can hide inside what looks like random bytes, and the hashing task proved that even one invisible byte like a stray newline is enough to break a file's integrity check. The timestamp work mattered the most to me: mixing up Unix time with Mac Absolute Time, or UTC with local time, could place an event decades or hours off from when it actually happened, which could seriously damage an investigation's credibility. The endianness exercise drove home a similar point identical raw bytes can produce two completely different numbers depending on the assumed byte order, which matters a lot when reading memory dumps or disk metadata. Overall, this lab taught me that accurate forensic interpretation comes down to precision at the byte level. As an examiner, I can't just trust how a value looks on screen I have to confirm the base, encoding, epoch, and byte order before treating anything as reliable evidence.